LAB 2: CRUD OPERATIONS IN MONGODB

Track	Data & AI
Role	Software Development - Backend
Level	Level 2
Course	NoSQL Databases
Scope	• Implement CRUD operations on telecom data in MongoDB. • Perform bulk data manipulation and advanced queries. • Apply atomic operations and transactions for data integrity.
Prerequisite	• Basic understanding of MongoDB and NoSQL databases. • MongoDB Compass and Mongo Shell installed on your system. • Familiarity with JSON data structure.

Objective

Effectively perform CRUD operations on telecom data using MongoDB Compass and Mongo Shell, utilize advanced data manipulation techniques like bulk operations, and ensure data integrity through atomic operations and transactions.

Tasks

Dataset: Given

1. **Create Operations:**
    - Insert the telecom dataset as a JSON file into MongoDB Compass.
    - Perform bulk insertion of the dataset into MongoDB using Mongo Shell.
2. **Read Operations:**
    - Query customer data in MongoDB Compass based on subscription region.
    - Use Mongo Shell to perform advanced queries, such as filtering customers with premium plans and high monthly bills.
3. **Insert Data Using Queries:**
    - Update individual customer records in MongoDB Compass (e.g., changing subscription plans).
    - Perform bulk updates on customer records using Mongo Shell.
4. **Delete Operations:**
    - Delete a specific customer record in MongoDB Compass.
    - Perform bulk deletion of customers based on specific criteria using Mongo Shell.
5. **Advanced CRUD Techniques:**

- Perform atomic updates, such as incrementing customer tenure, using Mongo Shell.
- Implement transactions to update customer data and insert billing records simultaneously.

Outcomes

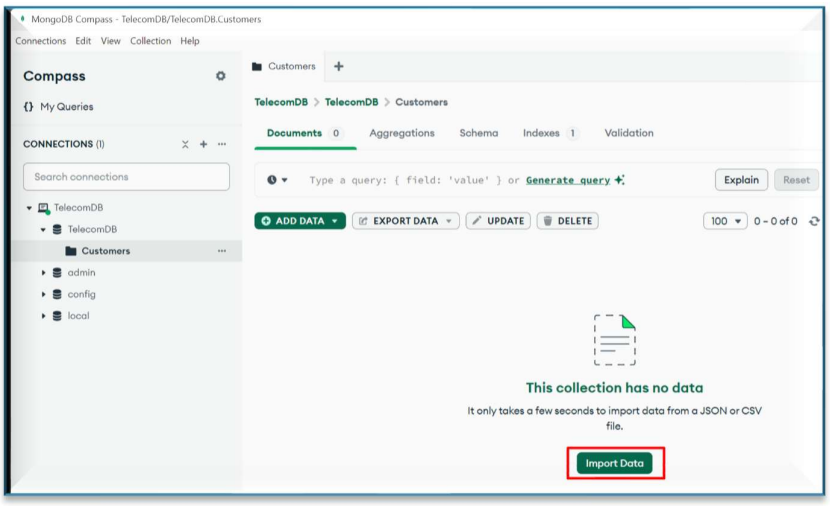
- Load and insert telecom data into MongoDB using Compass and Mongo Shell.
- Perform CRUD operations, including bulk inserts and atomic updates.
- Implement transactions to ensure data consistency.
- Gain real-world experience managing telecom data in MongoDB.

Solution

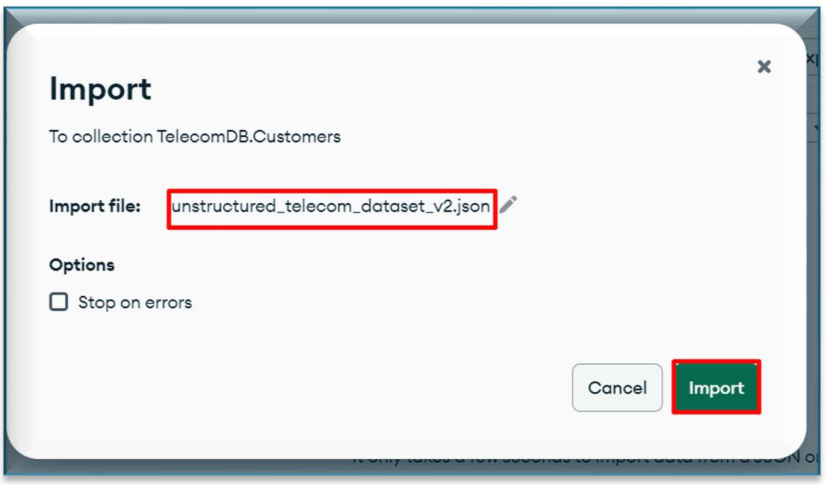
1. Create Operations:

a. Insert Dataset as JSON (MongoDB Compass):

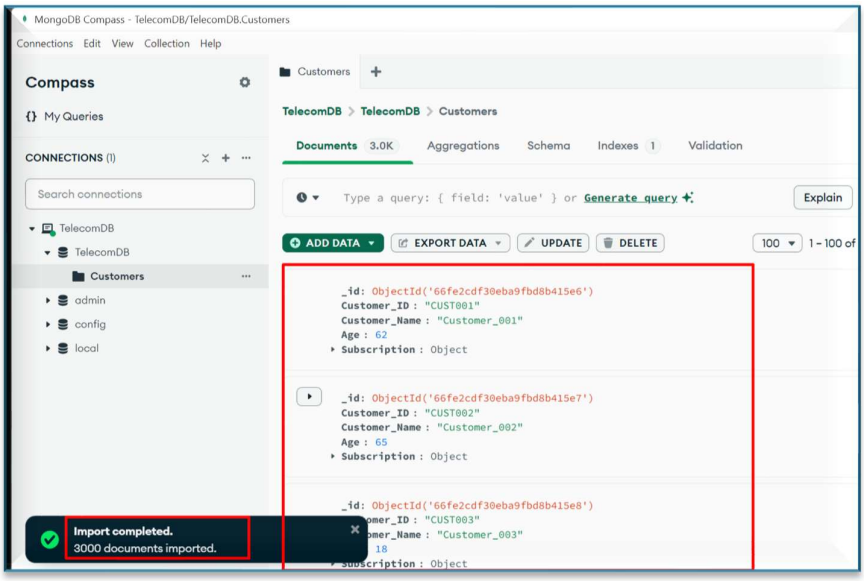
- Open MongoDB Compass and connect to your local MongoDB server.
- Navigate to the **TelecomDB** database. If it doesn't exist, create a new database called **TelecomDB**.
- Click on the Customers collection and select **Add Data**. (Creaet a new collection if it doesn't exist).
- In the dialog box, choose **Import JSON or CSV file**.
- Or you can click on **Import Data** button.



- Next Select the json file from your local system.
- And click on **Import** Button.



- Once data is imported successfully data will be visible in Customers collection.



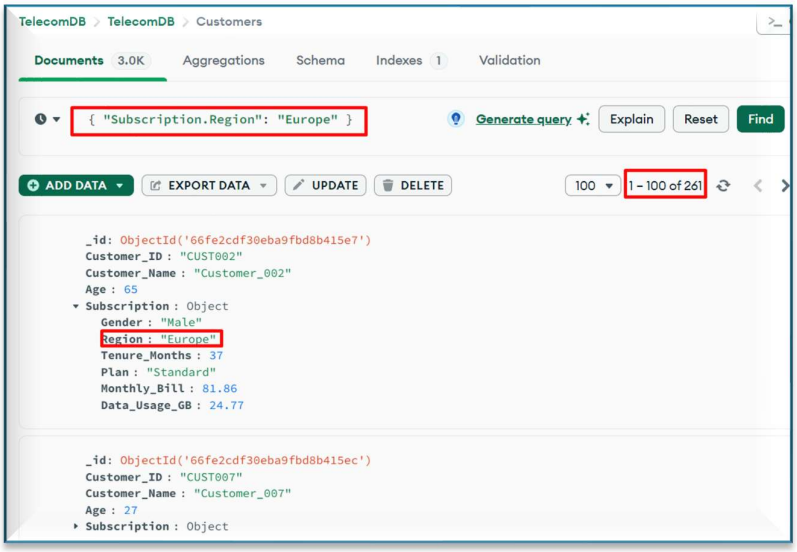
2. Read Operations:

a. Query Data (MongoDB Compass):

- Navigate to the Customers collection in MongoDB Compass.
- In the query bar, search for customers based on their subscription region. For example, to find all customers in Europe, use the following filter:

```
{ "Subscription.Region": "Europe" }
```

- Click **Find** to view the filtered results.



b. Query Data (MongoDB Compass):

- Use Mongo Shell to query customers with a Premium plan and a monthly bill greater than $80:

```
db.Customers.find({
    "Subscription.Plan": "Premium",
    "Subscription.Monthly_Bill": { $gt: 80 }
  });
```

- View the results directly in Mongo Shell.

```
>_MONGOSH
> use TelecomDB
< switched to db TelecomDB
> db.Customers.find({
    "Subscription.Plan": "Premium",
    "Subscription.Monthly_Bill": { $gt: 80 }
  });
< {
    _id: ObjectId('66fe2cdf30eba9fbd8b415e9'),
    Customer_ID: 'CUST004',
    Customer_Name: 'Customer_004',
    Age: 21,
    Subscription: {
      Gender: 'Male',
      Region: 'Australia',
      Tenure_Months: 86,
      Plan: 'Premium',
      Monthly_Bill: 88.42,
      Data_Usage_GB: 60.18
    }
  }
  {
    _id: ObjectId('66fe2cdf30eba9fbd8b415ed'),
```

- Find customers from "North America" who have a tenure of more than 50 months.

```
db.telecom.find({
    "Subscription.Region": "North America",
    "Subscription.Tenure_Months": { $gt: 50 }
});
```

- Output:

```
{
  _id: ObjectId('673c3ccaef1f7bf73fd5fd27'),
  Customer_ID: 'CUST001',
  Customer_Name: 'Customer_001',
  Age: 62,
  Subscription: {
    Gender: 'Male',
    Region: 'North America',
    Tenure_Months: 117,
    Plan: 'Premium',
    Monthly_Bill: 76.32,
    Data_Usage_GB: 54.58
  }
```

- Get customers with monthly bills greater than $80 sorted in descending order by their bill

```
db.telecom.find({ "Subscription.Monthly_Bill": { $gt: 80 } })
          .sort({ "Subscription.Monthly_Bill": -1 });
```

- Output:

```
  _id: ObjectId('673c3ccaef1f7bf73fd5fecd'),
  Customer_ID: 'CUST423',
  Customer_Name: 'Customer_423',
  Age: 53,
  Subscription: {
    Gender: 'Female',
    Region: 'Asia',
    Tenure_Months: 89,
    Plan: 'Standard',
    Monthly_Bill: 99.95,
    Data_Usage_GB: 96.91
  }
}
{
  _id: ObjectId('673c3ccaef1f7bf73fd5ffca'),
  Customer_ID: 'CUST676',
  Customer_Name: 'Customer_676',
  Age: 46,
  Subscription: {
    Gender: 'Male',
    Region: 'North America',
    Tenure_Months: 114,
    Plan: 'Premium',
    Monthly_Bill: 99.87,
    Data_Usage_GB: 35.42
```

- Find the first 5 customers who use more than 60GB of data

```
db.telecom.find({ "Subscription.Data_Usage_GB": { $gt: 60 } })
          .limit(5);
```

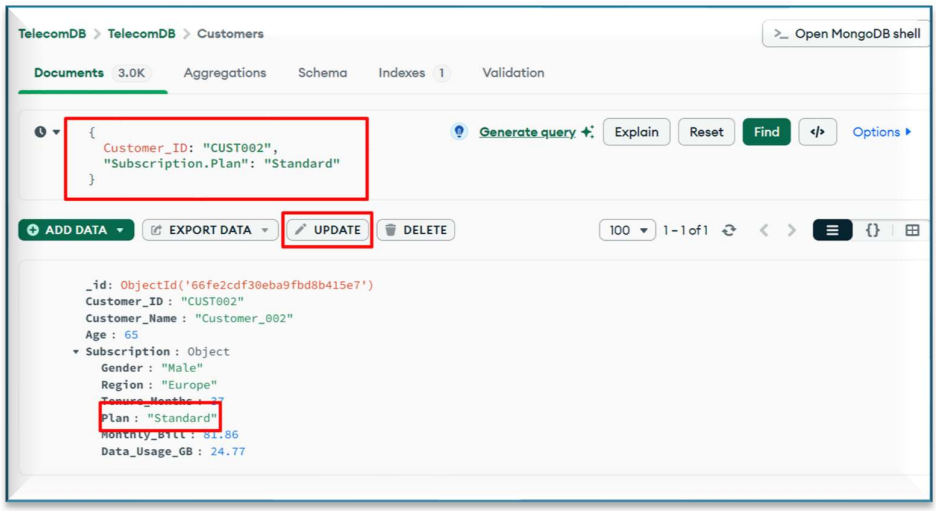
- Output:

```
{
  _id: ObjectId('673c3ccaef1f7bf73fd5fd2a'),
  Customer_ID: 'CUST004',
  Customer_Name: 'Customer_004',
  Age: 21,
  Subscription: {
    Gender: 'Male',
    Region: 'Australia',
    Tenure_Months: 86,
    Plan: 'Premium',
    Monthly_Bill: 88.42,
    Data_Usage_GB: 60.18
  }
}
{
  _id: ObjectId('673c3ccaef1f7bf73fd5fd2b'),
  Customer_ID: 'CUST005',
  Customer_Name: 'Customer_005',
  Age: 21,
  Subscription: {
```

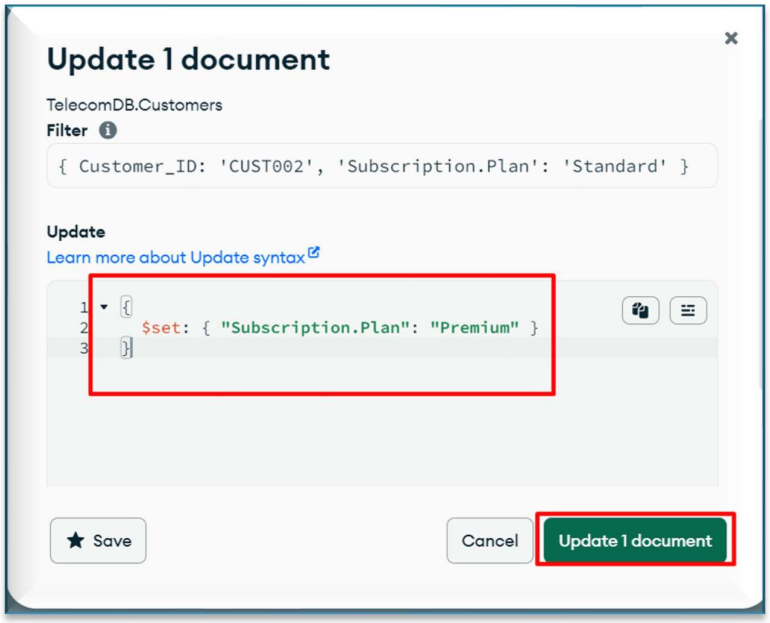
3. Update Individual Records (MongoDB Compass):

a. Update Individual Records (MongoDB Compass):

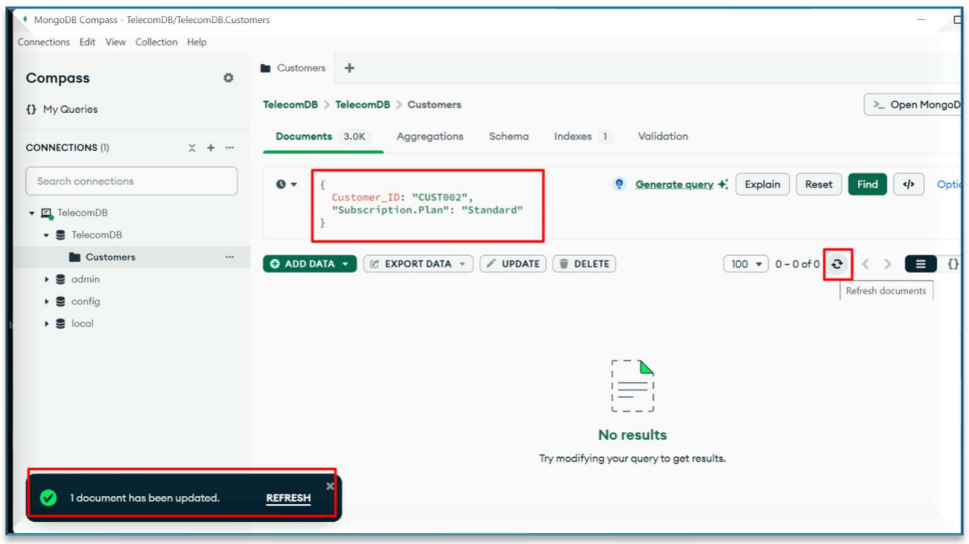
- In MongoDB Compass, go to the Customers collection.
- Search for a specific customer, Customer_ID "CUST002".



- Click on the document to open it, then click **Edit**.
- Modify the Subscription.Plan field from "Standard" to "Premium" and click **Update** to commit the changes.



- Once updated try to find the Subscription.Plan with "Standard"for Customer_ID "CUST002" to see any records.



- You can also update directly from MongoDB Shell.
- Update the "Plan" of all customers in the "Asia" region with a "Basic" plan to "Standard"

```
db.telecom.updateMany(
    { "Subscription.Region": "Asia", "Subscription.Plan": "Basic" },
    { $set: { "Subscription.Plan": "Standard" } }
);
```

- Output:

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 85,
  modifiedCount: 85,
  upsertedCount: 0
}
```

- Increase the "Monthly_Bill" by 10% for all customers who have used more than 70GB of data

```
db.telecom.updateMany(
    { "Subscription.Data_Usage_GB": { $gt: 70 } },
    { $mul: { "Subscription.Monthly_Bill": 1.1 } });
```

- Output:

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 307,
  modifiedCount: 307,
  upsertedCount: 0
}
```

b. Bulk Update (Mongo Shell):

- Use Mongo Shell to bulk update the subscription plan for all customers in the Asia region to "Premium":
- Verify the update by running a query to check the updated records.

```
db.Customers.updateMany(
  { "Subscription.Region": "Asia" },
  { $set: { "Subscription.Plan": "Premium" }
}
```

```
Customers    >_ mongosh: TelecomDB    +

>_MONGOSH

> use TelecomDB
< switched to db TelecomDB
> db.Customers.updateMany(
    { "Subscription.Region": "Asia" },
    { $set: { "Subscription.Plan": "Premium" } }
  );
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 248,
    modifiedCount: 175,
    upsertedCount: 0
  }
TelecomDB >
```

- Verify the update by running a query to check the updated records.

```
{ "Subscription.Region": "Asia",
"Subscription.Plan": { $ne: "Premium" } }
```

Generate query | Explain | Reset | Find | Op

ADD DATA | EXPORT DATA | UPDATE | DELETE | 100 | 0 – 0 of 0

No results

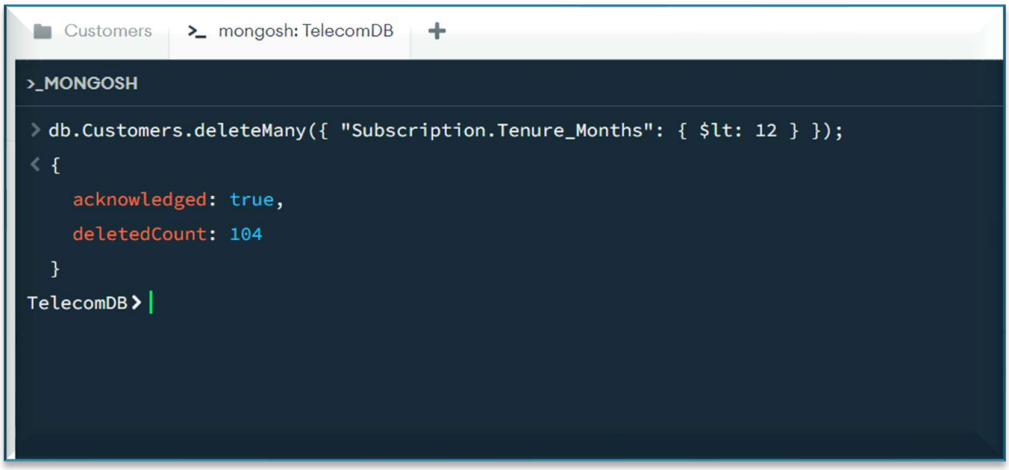
Try modifying your query to get results.

4. Delete Operations:

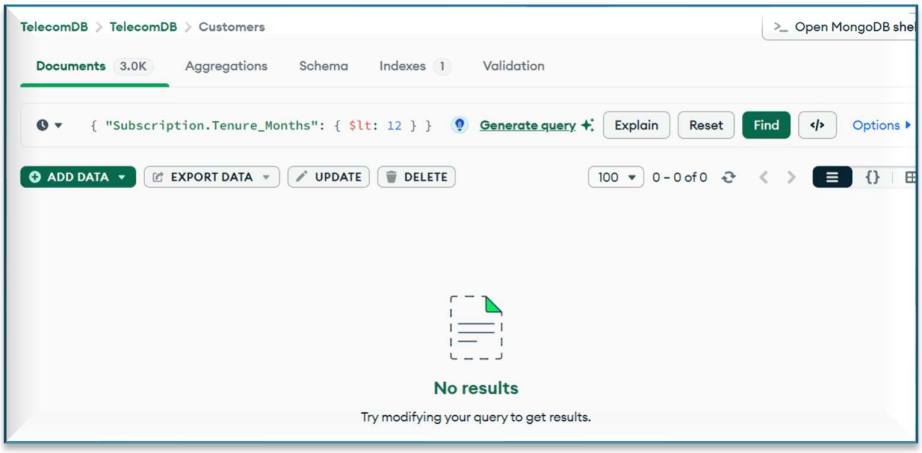
a. Bulk Delete (Mongo Shell):

- Use Mongo Shell to bulk delete customers with a tenure of fewer than 12 months:

```
db.Customers.deleteMany({ "Subscription.Tenure_Months": { $lt: 12 } });
```



- Confirm the deletion by querying the collection.



- Delete all customers with a tenure of less than 12 months

```
db.telecom.deleteMany({ "Subscription.Tenure_Months": { $lt: 12 } });
```

- Output:

```
{
  acknowledged: true,
  deletedCount: 104
}
```

- Delete a specific customer by their Customer_ID (e.g., CUST050)

```
db.telecom.deleteOne({ "Customer_ID": "CUST050" });
```

- Output:

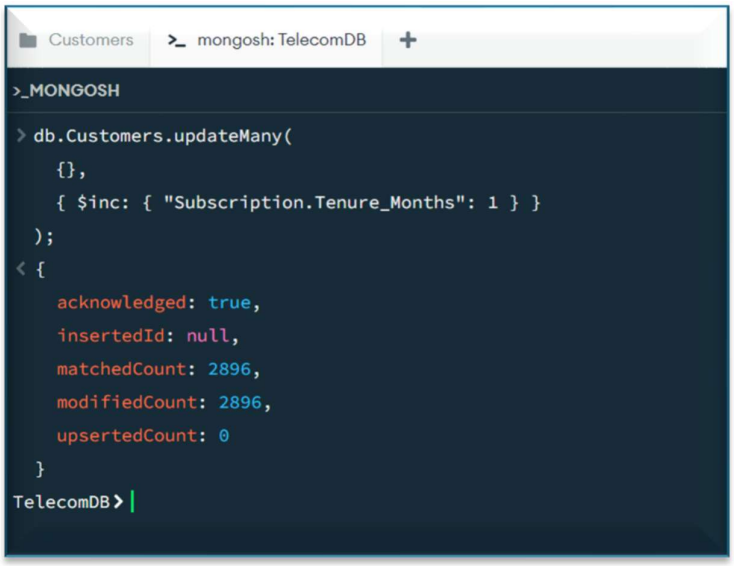
```
{
  acknowledged: true,
  deletedCount: 1
}
```

5. Advanced CRUD Techniques

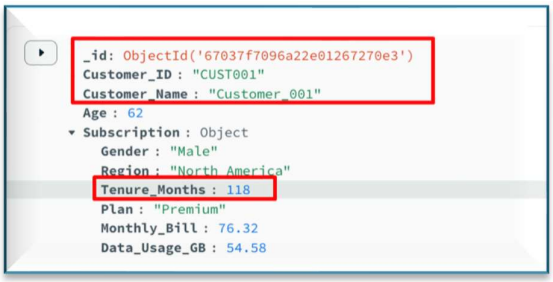
a. Atomic Operations:

- Increment the Tenure_Months field for all customers by 1 month:

```
db.Customers.updateMany(
  {},
  { $inc: { "Subscription.Tenure_Months": 1 } }
);
```



- Verify that all customers' Tenure_Months have been updated successfully.



b. Transactions Operations:

- Start a session in Mongo Shell and perform a transaction that updates a customer's billing information and inserts a record into a Billing collection. Both operations will either succeed or fail together.
- Starts a new session and begins a transaction.
- Updates the customer's billing information for Customer 001:
  I. Sets the new monthly bill to $80.00.
  II. Updates the plan to "Premium Plus".

```
// Start a session
const session = db.getMongo().startSession();

// Start a transaction
session.startTransaction();

try {
  // Get the collections
  const customersCollection =
session.getDatabase("TelecomDB").Customers;
  const billingCollection = session.getDatabase("TelecomDB").Billing;

  // Update customer's billing information
  const updateResult = customersCollection.updateOne(
    { Customer_ID: "CUST001" },
    {
      $set: {
        "Subscription.Monthly_Bill": 80.00,
        "Subscription.Plan": "Premium Plus"
      }
    }
  );

  console.log(`Updated ${updateResult.modifiedCount} customer
document(s)`);
```

- Inserts a new record into the Billing collection with the updated information.

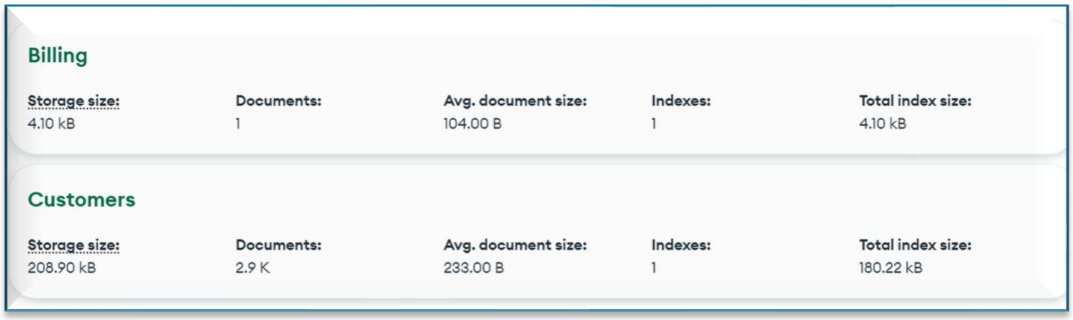
- If all operations are successful, it commits the transaction.
- If any error occurs, it aborts the transaction.
- Finally, it ends the session.

```
  // Insert a record into the Billing collection
  const billingInsertResult = billingCollection.insertOne({
    Customer_ID: "CUST001",
    Billing_Date: new Date(),
    Amount: 80.00,
    Plan: "Premium Plus"
  });

  console.log(`Inserted billing record with ID:
${billingInsertResult.insertedId}`);

  // Commit the transaction
  session.commitTransaction();
  console.log("Transaction committed successfully");
} catch (error) {
  // If an error occurred, abort the transaction
  session.abortTransaction();
  console.error("Transaction aborted. Error:", error);
} finally {
  // End the session
  session.endSession();
}
```

- Verify the transaction was successful by querying both the Customers and Billing collections.



Billing

Storage size:	Documents:	Avg. document size:	Indexes:	Total index size:
4.10 kB	1	104.00 B	1	4.10 kB

Customers

Storage size:	Documents:	Avg. document size:	Indexes:	Total index size:
208.90 kB	2.9 K	233.00 B	1	180.22 kB

(ADDITIONAL) TASK TO DO:

- Perform tasks mentioned above in this lab for below datasets:

  Dataset : json_datasets.rar